

计算机程序设计L习题课

杜晓阳 宋雨声 喻云山



中国科学技术大学
University of Science and Technology of China



01

2025年综合测试题



中国科学技术大学
University of Science and Technology of China



P1T5

5. 若 `int a[]={1,2,3,4,5,6,7,8,9,10}`, `*p=a`; 则值为 3 的表达式是 (A)。

- A. `p+=2, *p++` B. `p+=2, *++p`
C. `p+=3, *p++` D. `p+=2, ++*p`

`p+=2`: 指针 `p` 向后移动 2 位, 指向 `a[2]` (数值 3)。

`*p++`: 逗号表达式取最后一项的值。由于 `++` 是后置的, 先取值, 后移动。

B (`*++p`): 前置自增, 先移到下一位 (4), 再取值, 结果为 4。

C (`p+=3`): 直接移到了 `a[3]`。

D (`++*p`): 先取出 3 再 `++`, 结果为 4。

P1T6

6. 设 `int(*ptr)[10]`; 其中的 `ptr` 是 (**C**)。
- A. 10 个指向整型变量的指针
 - B. 指向 10 个整型变量的函数指针
 - C. 一个指向具有 10 个元素的一维数组的指针
 - D. 具有 10 个指针元素的整型数组

P1T6

理解 `int (*ptr)[10]` 的含义，关键在于**运算符的优先级**和**括号**的作用。

1. `(*ptr)` 这里的括号强制提高了优先权，说明 `ptr` 首先是一个**指针**。
2. 右边有 `[10]`，说明它指向的是一个**包含 10 个元素的数组**。
3. 左边是 `int`，说明数组的元素是**整型**。

- **C. 数组指针**

- **写法**: `int (*ptr)[10];`
- **含义**: `ptr` 是一个**指针**，指向一个长为 10 的数组。

- **A/D.**

- **写法**: `int *ptr[10];`
- **含义**: 因为 `[]` 优先级比 `*` 高，所以 `ptr` 首先是一个**数组**。它有 10 个位置，每个位置存的都是一个 `int *`（整型指针）。

P2T10

10. 以下调用 scanf 函数给变量 a 输入数值的方法是错误的，其原因是（ B ）。

```
int main() {  
    int a, *p, *q, b; p = &a;  
    printf("input a:");  
    scanf("%d", *p);  
    .....  
}
```

- A. *p 表示的是指针变量 p 的地址
- B. *p 表示的是变量 a 的值，而不是变量 a 的地址
- C. *p 表示的是指针变量 p 的值
- D. *p 只能用来说明 p 是一个指针变量

P2T10

- A. *p 表示的是指针变量 p 的地址
 - 错误。指针变量 p 自己的地址应该是 &p。
- B. *p 表示的是变量 a 的值，而不是变量 a 的地址
 - 正确。scanf 需要地址 (p 或 &a)，但你给了它值 (*p 或 a)。
- C. *p 表示的是指针变量 p 的值
 - 错误。指针变量 p 的“值”是它存储的地址 (即 &a)。*p 是它指向的内容。
- D. *p 只能用来说明 p 是一个指针变量
 - 错误。虽然在定义变量时用来标识指针类型，但在执行语句中 (如 scanf 里)，* 是运算符，用来取出指针指向的数据。

P2T5

5. 设 ch 是 char 型变量, 其值为 'A', 则执行语句

`ch = (ch >= 'A' && ch <= 'Z') ? (ch + 32) : ch;`后, ch 的值为 97。

这条语句的作用是将大写字母转换为小写字母。执行后, ch 的值变为 'a'。(填'a', 97都可以)

ASCII表有几个值需要记住:

'A':65

'a':97

'0':48

P3T3

3.

```
#include <stdio.h>
int n = 2;
int fun(int n) {
    int a = 1;
    static int m = 5;
    m = n + m + a;
    printf("%4d%4d\n", n, a);
    return (m);
}

int main() {
    int a = 1;
    printf("%4d\n", fun(n + fun(a)));
    return 0;
}
```

P3T3

考察 C 语言中的变量作用域 以及 静态局部变量 的特性。

1. 函数 fun 内的变量:

- `int a = 1;`: 普通的局部变量, 每次调用 `fun` 都会重新初始化为 1。
- `static int m = 5;`: **静态局部变量**。它只在第一次调用时初始化一次, 之后它的值会一直保留到下一次调用。

2. 执行过程

这里的执行顺序是: 先计算内层的 `fun(a)`, 再计算外层的 `fun(n+fun(a))`, 最后执行 `printf`。

P3T3

第一步：执行内层函数 fun(a)

- 进入 fun(1):
 - 参数 `n = 1`。
 - 局部变量 `a = 1`。
 - 静态变量 `m = 5`。
- 计算:
 - `m = 1 + 5 + 1 = 7`
 - 此时静态变量 `m` 的值变成了 7。
- 输出:
 - `printf("%4d%4d\n", n, a);`
 - 输出: 1 1 (`%4d` 表示宽度为 4, 右对齐)。
- 返回:
 - `return (m);` 返回 7。

第二步：准备外层函数的参数

- 回到 `main` 函数, 表达式变成了 `fun(n + 7)`。
- 这里的 `n` 是全局变量, 值为 2。

第三步：执行外层函数 fun(9)

- 进入 fun(9):
 - 参数 `n = 9`。
 - 局部变量 `a = 1` (重新初始化)。
 - 静态变量 `m = 7` (保留了上一次调用后的值)。
- 计算:
 - `m = 9 + 7 + 1 = 17`
 - 此时静态变量 `m` 的值更新为 17。
- 输出:
 - `printf("%4d%4d\n", n, a);`
 - 输出: 9 1。
- 返回:
 - `return (m);` 返回 17。

第四步：执行 main 函数的输出

- `main` 函数执行 `printf("%4d\n", 17);`。

屏幕上将会按顺序显示三行内容：

```
1  1
9  1
17
```

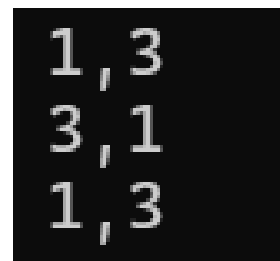
P4T6

```
6.
#include <stdio.h>
#define SWAP(x, y) temp = x; x = y; y = temp
void swap(int *p, int *q)
{
    int temp;
    temp = *p; *p = *q; *q = temp;
}
int main() {
    int a = 1, b = 3, *p = &a, *q = &b, *temp;
    printf("%d,%d\n", *p, *q);
    SWAP(p, q);
    printf("%d,%d\n", *p, *q);
    swap(p, q);
    printf("%d,%d\n", *p, *q);
    return 0;
}
```

这段代码中通过两种方式实现数据交换：宏定义和函数

其中宏定义是直接将主函数中的SWAP(p, q); 直接替换为temp = x; x = y; y = temp;这三条语句。

swap函数则是通过改变地址中存储的整数的值实现交换。



```
1, 3
3, 1
1, 3
```

P7T1、2、3、4

1. 计算机可以进行自动处理的基础是 ()。
A. 存储程序 B. 快速运算 C. 计算精度高 D. 能进行逻辑判断
2. 计算机的通用性使其可以求解不同的算术和逻辑问题, 这主要取决于其 ()。
A. 高速运算 B. 指令系统 C. 可编程性 D. 存储功能
3. 在计算机内部用于表示数据和指令的是 () 编码。
A. 十进制码 B. 二进制码 C. ASCII 码 D. 汉字编码
4. 计算机进行数值计算时的高精度主要取决于 ()。
A. 计算速度 B. 内存容量 C. 外存容量 D. 基本字长

P7T1、 2

1. 计算机可以进行自动处理的基础是（ A ）。

A. 存储程序

解析：这是著名的“冯·诺依曼体系结构”的核心思想。计算机之所以能自动运行，是因为我们将所有的指令（程序）和数据预先存储在计算机的内存中。启动后，控制器会按照地址顺序自动地从内存中取出指令并执行，中间不需要人工干预。

B、C、D 都是计算机的特点或能力，但不是实现“自动处理”的根本原因。

2. 计算机的通用性使其可以求解不同的算术和逻辑问题，这主要取决于其（ C ）。

C. 可编程性

解析：“通用性”指的是计算机不是只能做一件事（比如像老式计算器或专门的洗衣机控制器），而是可以通过改变软件来完成各种不同的任务。

可编程性意味着我们可以通过编写不同的程序（软件）来控制硬件，从而解决不同的问题。

A 是速度，B 是计算机能读懂的命令集合，D 是存储能力，这些都是基础，但只有“可编程性”直接解释了为什么它能解决“不同”的问题。

P7T3、 4

3. 在计算机内部用于表示数据和指令的是（ B ）编码。

B. 二进制码

解析： 计算机内部的电子元件（如晶体管）主要只有两种状态：通电（开）和断电（关），也就是高电平和低电平。这物理特性决定了计算机内部只能识别和处理由 `0` 和 `1` 组成的二进制代码。

无论是数字、文字（ASCII/汉字编码）还是程序的指令，在计算机最底层都是以二进制形式存在的。

4. 计算机进行数值计算时的高精度主要取决于（ D ）。

D. 基本字长

解析：

字长（Word Length）是指计算机一次能直接处理的二进制数据的位数（例如 32位、64位）。

字长越长，计算机在硬件层面能表示的数值范围就越大，有效数字的位数就越多，计算的精度也就越高。

A 决定快慢，B 和 C 决定能存多少东西，只有 D 直接关系到单个数值的精确度。

P7T5

5. 已知字母 A 的 ASCII 码为十进制数 65，且 c2 为字符型，则执行以下语句 `c2 = 'A' + '6' - '3';` 后，c2 中的值为 (B)。

A. D B. 68 C. 不确定 D. 'C'

'A' -> 65 '6' -> 54 '3' -> 51

$c2 = 65 + 54 - 51 = 68$

或者'6'与'3'的ASCII值相差3，因此 $c2 = 65 + 3 = 68$

为什么c2中的值不是'D'？

因为在C语言中，字符型变量(char)在内存中存储的永远是字符的ASCII码值(数值)，与是否输出无关。

输出只是将这个数值按字符格式展示的“翻译过程”。

例如：

```
int main() {  
    char c = 'D';  
    printf("%d\n", c);  
    printf("%c\n", c);  
  
    return 0;  
}
```

68
D

P7T10

10. C程序由函数组成，关于 C 语言函数的说法错误的是（ D ）

A. 除整型函数外，其他类型函数必须在定义时给以类型说明。—— 正确（基于传统 C 语言标准

如果函数定义时没有显式写出返回类型，编译器会默认该函数返回 int。返回非整型（如 float, void, char 等）的函数必须显式指明类型。

B. 函数原型说明语句必须给出函数类型的说明。—— 正确

函数原型的主要作用就是告诉编译器函数的名称、返回类型以及参数列表。

C. 函数类型决定返回值的类型。—— 正确

函数的返回值类型最终是由函数定义头部的类型决定的。如果 return 语句中的表达式类型与函数定义类型不一致，系统会自动进行类型转换（例如将 float 转为 int），以函数定义的类型为准。

D. 函数不仅可以递归定义，而且除 main 函数外都可以递归调用。—— 错误

递归定义是指在一个函数里面再定义自己，这是 C 不允许的，所有的函数定义必须是“平行”的，即一个函数不能包含另一个函数。

P8T5、6、7

5. 设已有 `int x = 3, y = 4`; 说明, 表达式 `(x = ++y, x + 5, x / 5)` 的值为: 1。

6. 设已有 `int j = 5, k = 2, n, m, p`; 说明, 执行以下三条语句后 `n, m` 和 `p` 的值是什么?

`n = j - ++k;    m = j-- + k--;    p = k + j;`

解: `n` 的值是 2, `m` 的值是 8, `p` 的值是 6。

7. 设已有 `int x = 3, y = 5, z = 2`; 说明, 执行以下三条语句后 `x, y` 和 `z` 的值是什么?

`x *= y + z;    y /= 2 * z + 1;    z += x;`

解: `x` 的值是 21, `y` 的值是 1, `z` 的值是 23。

P8T6

6.

1. 初始状态: $j = 5, k = 2$

2. 执行 $n = j - ++k$;

$++k$ 是前置自增: 先将 k 加 1, k 变为 3, 然后再参与计算。

表达式变为 $n = 5 - 3$ 。

结果: $n = 2$ 。

当前变量值: $j = 5, k = 3, n = 2$

3. 执行 $m = j-- + k--$;

$j--$ 和 $k--$ 都是后置自减: 先使用变量当前的值参与运算, 运算结束后再减 1。

运算取值: j 取 5, k 取 3。

表达式变为 $m = 5 + 3$, 所以 $m = 8$ 。

副作用 (自减): 运算后 j 变为 4, k 变为 2。

当前变量值: $j = 4, k = 2, n = 2, m = 8$

4. 执行 $p = k + j$;

此时 j 是 4, k 是 2。

表达式变为 $p = 2 + 4$ 。

结果: $p = 6$ 。

P8T7

7.

1. 初始状态: $x = 3, y = 5, z = 2$

2. 执行 $x *= y + z$;

赋值运算符 $*$ 的优先级低于 $+$, 所以右边先算。

等价于 $x = x * (y + z)$ 。

计算: $x = 3 * (5 + 2) = 3 * 7 = 21$ 。

当前变量值: $x = 21, y = 5, z = 2$

3. 执行 $y /= 2 * z + 1$;

同样, $/=$ 优先级最低, 右边先算完。

等价于 $y = y / (2 * z + 1)$ 。

计算: $y = 5 / (2 * 2 + 1) = 5 / 5 = 1$ 。

当前变量值: $x = 21, y = 1, z = 2$

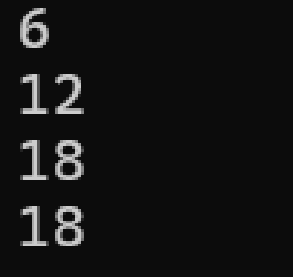
4. 执行 $z += x$;

等价于 $z = z + x$ 。

计算: $z = 2 + 21 = 23$ 。

P9T4

```
#include <stdio.h>
int multiply(int m, int n) {
    int ans; // 用于存储当前层级的计算结果
    if (n == 1) {
        ans = m; // 当 n 减小到 1 时, 结果就是 m 本身
    }
    else { // --- 递归调用
        // 核心逻辑:  $m * n = m + m * (n-1)$ 
        ans = m + multiply(m, n - 1);
    }
    printf("%d\n", ans); // 这行代码是在递归调用返回之后执行的
    return ans; // 将结果返回给上一层调用者
}
int main() {
    printf("%d\n", multiply(6, 3)); // 调用函数, 并打印最终的返回值
    return 0;
}
```



6
12
18
18

P12T11

11. 有数组定义和函数 fun 调用语句 `int a[3][4]; fun(a);`，则在函数 fun 定义时，对形参 array 的错误定义方式为（ C ）。

A. `fun(int array[][4])`

B. `fun(int array[3][4])`

C. `fun(int **array)`

D. `fun(int (*array)[4])`

在 C 语言中，数组名作为函数参数时会被“退化”为指针。对于二维数组 `a[3][4]`，当传入函数时：a 会退化为指向第一行的指针，即 `int (*)[4]`（指向一个长度为 4 的整型数组的指针）。

C 选项中 `**array` 表示 array 是一个指向指针的指针（二级指针），与二维数组 `int a[3][4]` 的内存布局 and 类型不匹配，不能直接传递。

P12T12

12. 以下选项中，操作数必须是整型或字符型的运算符是 (C)。

A. ++

B. !

C. %

D. /

如果 '%' 的操作数是字符类型（例如 'A'、'5' 等），实际使用的是它们的ASCII值

```
int main() {  
    char ch = 'A'; // ASCII 码为 65  
    int result = ch % 3;  
    printf("'A' %% 3 = %d\n", ch, result);  
    return 0;  
}
```

'A' % 3 = 2

'++' 可作用于整型和指针型

'!' 可作用于任意算术类型（整型、浮点型）和指针型

'/' 可作用于整型和浮点型

P13T16

16. 若有语句 `int *point, a=4; point=&a;` 下面均代表地址的一组选项是 (C)。

A. `a, point, *&a`

B. `&*a, &a, *point`

C. `&a, &*point, &point`

D. `*&point, *&*point, &a`

B 选项中 '`&*a`' 是非法的, 因为 `*` 是解引用运算符, 它要求操作数是一个指针类型。

D 选项中 '`*&*point`' 即 '`*(&(*point))`', 其中 `*point` 代表对变量 `a` 的访问; `&(*point)` 代表变量 `a` 的地址, 即 `&a`, 等价于 `point`; `*(&(*point))` 对地址解引用, 因此表达式代表变量 `a` 的值, 即 4。

P13T18

18. 若有程序如下:

```
#include <stdio.h>
void swap(int* x, int* y){
    int *t;
    t=x, x=y, y=t;
}
int main(){
    int a=3, b=4;
    swap(&a, &b);
    printf("%d,%d", a, b);
    return 0;
}
```

则程序的输出为 (**B**)。

A. 3,3

B. 3,4

C. 4,4

D. 4,3

swap(&a, &b) 将 a 和 b 的地址传给函数。所以在 swap 函数中, x 指向 a (即 $x = \&a$) ; y 指向 b (即 $y = \&b$)

注意: 函数只交换了 指针 x 和 y, 即

x 原本指向 a, 现在指向 b

y 原本指向 b, 现在指向 a

但 a 和 b 的值本身没有改变

如果想交换a和b的值可以修改为:

```
void swap(int* x, int* y) {
    int t;
    t = *x, *x = *y, *y = t;
}
```

P14T22

22. 若有：int a[4][5], *p=*a;, 则以下选项中可以表示a[0][3]的是 (B)。

A. p[0][3] B. p[3] C. **(a+3) D. **a+3

A. p[0][3]

p 是一级指针 (int*)，p[0] 已经是取出的一个整数值了，即a[0][0]，不能对“整数值”再用下标[3]。只有二级指针或二维数组名才能用双下标。

B. p[3] —— 【正确】

p 把二维数组看作一根线性的一维数组。从开头 a[0][0] 往后数 3 个位置，正好是 a[0][3]。

C. **(a+3)

a 是二维数组名，看作指向行的指针（行指针）。a+3 是跳过 3 行，指向第 4 行的开头，即 a[3][0]，而不是第 0 行的第 3 个。

D. **a+3

a 取出的是 a[0][0] 的数值。a+3 表示 a[0][0]+3 的值。

P19T3

3. 将两个 8 位有符号数运算后的结果按 8 位无符号数处理其值为 160，实际上它代表 (B)
- A) -95 B) -96 C) -128 D) -160

由于 $2^7 = 128 < 160$ ，因此符号位是 1（原值为负），且它的绝对值为 $160 - 128 = 32$ ，这个二进制数是 10100000，而负数以补码方式储存，除去符号位（0100000）后先取反得到 1011111，再 +1 得到 1100000，即 96，因此原本的二进制数实际代表 -96。

P19T6

6. 设有语句：int a=2,b=3,c=4; float x=3.5,y=4.8;

则表达式 $!(a+b)+c-1 \&\&b+c/2$ 和表达式 $x+a\%3*(int)(x+y)\%2/4$ 的值分别为 (B)

A) 0和3.50000

B) 1和3.50000

C) 0和4.50000

D) 1和4.50000

逻辑运算中，所有非0取反均为0，0取反为1。&&（与运算符）两侧均为真（非0）时为1，否则为0。

$x + y = 3.5 + 4.8 = 8.3$

$(int)(x+y)$ ：强制类型转换，将 8.3 截断为整数 8。

P22T18

18. 下列语句中，将 p 定义成一个指针型变量的是 (B)。

A) `double *p[5];`

B) `double (*p)[5];`

C) `double *p(5);`

D) `double *p();`

A 选项定义了一个指针数组，每个元素的类型是 `double *`

B 选项定义了一个数组指针

C 选项的表达式非法

D 选项声明了一个函数，其返回值类型是 `double *`

P24T4

4. 若有说明语句如下:

```
char a[]="It is mine";
```

```
char *p=a;
```

则以下正确的叙述是 (B C)

- A) a+2表示的是字符' t' 所在存储单元的地址
- B) p 指向另外的字符串时, 字符串的长度不受限制
- C) *(p+i)等价于p[i]
- D) a中只能存放10个字符

A:a+1 是 &a[1] (字符 't' 的地址) , a+2 是 &a[2] (字符空格 ' ' 的地址) 。

B:数组 a 在定义时, 大小就已经固定了; 指针 p 是一个指针变量, 通过改变指针 p 的指向, 它可以引用任意长度的字符串。

C:对于指针或数组, 表达式 p[i] 在编译时本质上就是被转换为 *(p+i) 来处理的, 两者完全等价。

D:编译器在初始化数组 a 时, 会分配 11 个字节的空间 (10 个字符 + 1 个 \0) 。

P25T3

3. 数组a中存放N个由小到大排列的有序整数。把从键盘输入的整数m插入到数组a中，使插入后的数组a仍然有序，请填空。

```
#define N 6
```

```
int main() {
```

```
int i,j,m;
```

```
int a[__(9)__]={10,20,30,40,50,60};
```

```
scanf( "%d" ,&m);
```

```
for(j=0;j<N;j++)
```

```
    if(__(10)__<a[j]) break;
```

```
for(i=N;i>j;i--)
```

```
    a[i]=a[__(11)__];
```

```
a[j]=__(12__);
```

```
for(i=0;i<N+1;i++)
```

```
    printf( "%d" ,a[i]);
```

```
}
```

(9) N+1

(10) m

(11) i-1

(12) m

P24T9

9. 若有 $\text{int } i=1, a[10][5]=\{1,2,3,4,5,6,7,8,9,10\}, (*p)[5]=a;$ 则表达式 $*(*(p+i)+5)$ 的值为 (10) 0

初始化时按行优先顺序填入数据

第 0 行: $a[0][0]=1, a[0][1]=2, a[0][2]=3, a[0][3]=4, a[0][4]=5$

第 1 行: $a[1][0]=6, a[1][1]=7, a[1][2]=8, a[1][3]=9, a[1][4]=10$

第 2~9 行: 未显式初始化, 全为 0

$(*p)[5]$ 表示 p 是一个指向包含 5 个整数的数组的指针

$*p + i$ 即 $\&a[0][0] + 1$, 表示指向 $a[1][0]$ 的地址 (即第二行第一个元素)

$(*p + i) + 5$ 即 $\&a[1][0] + 5$, 得到的是从 $a[1][0]$ 开始偏移 5 个 int 的地址, 即 $\&a[1][0] + 5$ 。

由于整个二维数组在内存中是连续存储的, 该地址恰好等于 $\&a[2][0]$

所以 $*((*p + i) + 5) = *(\&a[2][0]) = a[2][0] = 0$

```

void change(int *s[], int n) { // 实现矩阵转置
    int i, j, t;
    for(i = 0; i < 4; i++)
        for(j = i; j < 4; j++) {
            t = *(s[j] + i); // *(s[j] + i) 即矩阵的第 j 行, 第 i 列的值
            *(s[j] + i) = *(s[i] + j);
            *(s[i] + j) = t;
        }
}

main() {
    int a[4][4] = {
        {1, 2, 3, 4},
        {12, 13, 14, 5},
        {11, 16, 15, 6},
        {10, 9, 8, 7}
    };

    int i, j, *p[4]; // p 是一个 "行指针数组"
    for(i = 0; i < 4; i++)
        p[i] = a[i]; // p[i] 指向 a[i] 的首地址

    change(p, 4);

    for(i = 0; i < 4; i++)
        for(j = 0; j < 4; j++)
            printf("%2d%c", *(p[i] + j), j<3 ? ' ' : '\n');
}

```

1	12	11	10
2	13	16	9
3	14	15	8
4	5	6	7

p 是一个“行指针数组”，其中每个元素分别指向 a 中对应行的首地址

change 函数实现矩阵转置，其中 s[j] 指代 a 的第 j 行的首地址，s[j] + i 指代第 j 行，第 i 列元素的地址，因此 *(s[j] + i) 即矩阵的第 j 行，第 i 列的值

谢谢！



中国科学技术大学
University of Science and Technology of China

